

國立臺灣大學 National Taiwan University

Lab0

EE 5173 Operating System

TA: Hsueh-Yi, Chen. Advisor: Farn, Wang.

2025/3/5

Outline

- Pintos Details
- Environment Setup
- Report & Policy

Let’s dive into Pintos!

Source Tree Overview

- **threads/** : Source code for the base kernel, which you will modify starting in lab 2.
- **userprog/**: Source code for the user program loader, which you will modify starting with lab 1.
- **vm/**: An almost empty directory. You will implement virtual memory here in lab 3.
- **filesys/**: Source code for a basic file system. You will use this file system starting with lab 2, but you will not modify it until lab 4.

Source Tree Overview

- **devices/** : Source code for I/O device interfacing: keyboard, timer, disk, etc. You will modify the timer implementation in lab 2. Otherwise you should have no need to change this code.
- **lib/**: An implementation of a subset of the standard C library. The code in this directory is compiled into both the Pintos kernel and, starting from lab 1, user programs that run under it. In both kernel code and user programs, headers in this directory can be included using the #include <...> notation. You should have little need to modify this code.
- **lib/kernel/**: Parts of the C library that are included only in the Pintos kernel. This also includes implementations of some data types that you are free to use in your kernel code: bitmaps, doubly linked lists, and hash tables. In the kernel, headers in this directory can be included using the #include <...> notation.

Source Tree Overview

- **lib/user/**: Parts of the C library that are included only in Pintos user programs. In user programs, headers in this directory can be included using the #include <...> notation.
- **tests/**: Tests for each project. You can modify this code if it helps you test your submission, but we will replace it with the originals before we run the tests.
- **examples/**: Example user programs for use starting with lab 1.
- **misc/, utils/**: These files may come in handy if you decide to try working with Pintos on your own machine. Otherwise, you can ignore them.

Pintos Development

- Build your kernel code
- Run pintos
- Run tests & debug

Building Pintos

- cd into the threads directory
- Issue the make command. This will create a build directory under thread

```
root@8931b75d6a21:/home/pintos/threads/build# ls
Makefile  devices  kernel.bin  kernel.o  lib  loader.bin  tests  threads
```

Building Pintos

- **Makefile**: A copy of pintos/src/Makefile.build. It describes how to build the kernel.
- **kernel.o**: Object file for the entire kernel. This is the result of linking object files compiled from each individual kernel source file into a single object file. It contains debug information, so you can run GDB or backtrace on it.
- **kernel.bin**: Memory image of the kernel, that is, the exact bytes loaded into memory to run the Pintos kernel. This is just kernel.o **with debug information stripped out**, which saves a lot of space, which in turn keeps the kernel from bumping up against a 512 kB size limit imposed by the kernel loader's design.
- **loader.bin**: Memory image for the kernel loader, a small chunk of code written in assembly language that reads the kernel from disk into memory and starts it up. It is exactly 512 bytes long, a size fixed by the PC BIOS.

Running Pintos

- There is a program running Pintos in emulator : **pintos**
- Invoking**: pintos argument**
- Example: pintos **run alarm-multiple**
- Option Setting: **pintos option... -- argument....**

  pintos -v -- run alarm-multiple

Testing

- cd into different directory's /build directory, and type **make check**
- Test t writes its output to t.output, script scores it as "pass" or "fail" and writes it to t.result
- Individual tests one at a time (current directly is in threads/build/)
  - E.g, **make tests/threads/alarm-multiple.result**
  - If make says that the test result is up-to-date, but you want to re-run it anyway, either run **make clean** or **delete the .output file** by hand
- Observe the progress: **make check VERBOSE=1**
- You can write you own tests in the test directory of root directory for your purpose, but we will replace to get your scores.

Let’s do it!

Environment Setup

- **VScode** : https://code.visualstudio.com/download
- **Docker** : https://docs.docker.com/get-started/get-docker/
- **Github** : https://desktop.github.com/download/

Building Pintos

- git clone git@github.com:Xueyi-Chen-David/pintos.git
- docker run -it --rm --name pintos --mount type=bind,source=[absolute/path/to/pintos/on/your/host/machine],target=/home/pintos davidchen871/pintos bash

Target don't change it

- **docker compose up -d**
- **docker exec -it pintos bash**

either (docker compose recommend)

```
version: '3.8'

▷Run All Services
services:
  ▷Run Service
  pintos:
    image: davidchen871/pintos
    container_name: pintos
    stdin_open: true
    tty: true
    restart: no
    #volumes:
    # - [your_source_code_path]:/home/pintos
    environment:
      - PINTOS_DIR=/home/pintos
```

Uncomment volumes and set [your source code path] to absolute path of pintos repo

- cd /home/pintos/threads/
- make
- cd build

Test If Success

- pintos --qemu -- -q run alarm-multiple

```
(alarm-multiple) thread 4: duration=50, iteration=2, product=100
(alarm-multiple) thread 1: duration=20, iteration=6, product=120
(alarm-multiple) thread 2: duration=30, iteration=4, product=120
(alarm-multiple) thread 3: duration=40, iteration=3, product=120
(alarm-multiple) thread 1: duration=20, iteration=7, product=140
(alarm-multiple) thread 4: duration=50, iteration=3, product=150
(alarm-multiple) thread 2: duration=30, iteration=5, product=150
(alarm-multiple) thread 3: duration=40, iteration=4, product=160
(alarm-multiple) thread 2: duration=30, iteration=6, product=180
(alarm-multiple) thread 3: duration=40, iteration=5, product=200
(alarm-multiple) thread 4: duration=50, iteration=4, product=200
(alarm-multiple) thread 2: duration=30, iteration=7, product=210
(alarm-multiple) thread 3: duration=40, iteration=6, product=240
(alarm-multiple) thread 4: duration=50, iteration=5, product=250
(alarm-multiple) thread 3: duration=40, iteration=7, product=280
(alarm-multiple) thread 4: duration=50, iteration=6, product=300
(alarm-multiple) thread 4: duration=50, iteration=7, product=350
(alarm-multiple) end
Execution of 'alarm-multiple' complete.
Timer: 595 ticks
Thread: 0 idle ticks, 595 kernel ticks, 0 user ticks
Console: 2954 characters output
Keyboard: 0 keys pressed
Powering off...
```

Report (Lab0)

- f10921a18.png (.jpg)

```
(alarm-multiple) thread 1 sleeps 20 ticks each time, and so on.
(alarm-multiple) If successful, product of iteration count and
(alarm-multiple) sleep duration will appear in nondescending order.
(alarm-multiple) thread 0: duration=10, iteration=1, product=10
(alarm-multiple) thread 0: duration=10, iteration=2, product=20
(alarm-multiple) thread 1: duration=20, iteration=1, product=20
(alarm-multiple) thread 0: duration=10, iteration=3, product=30
(alarm-multiple) thread 2: duration=30, iteration=1, product=30
(alarm-multiple) thread 0: duration=10, iteration=4, product=40
(alarm-multiple) thread 1: duration=20, iteration=2, product=40
(alarm-multiple) thread 3: duration=40, iteration=1, product=40
(alarm-multiple) thread 4: duration=50, iteration=1, product=50
(alarm-multiple) thread 0: duration=10, iteration=5, product=50
(alarm-multiple) thread 2: duration=30, iteration=2, product=60
(alarm-multiple) thread 0: duration=10, iteration=6, product=60
(alarm-multiple) thread 1: duration=20, iteration=3, product=60
(alarm-multiple) thread 0: duration=10, iteration=7, product=70
(alarm-multiple) thread 1: duration=20, iteration=4, product=80
(alarm-multiple) thread 3: duration=40, iteration=2, product=80
(alarm-multiple) thread 2: duration=30, iteration=3, product=90
(alarm-multiple) thread 1: duration=20, iteration=5, product=100
(alarm-multiple) thread 4: duration=50, iteration=2, product=100
(alarm-multiple) thread 1: duration=20, iteration=6, product=120
(alarm-multiple) thread 2: duration=30, iteration=4, product=120
(alarm-multiple) thread 3: duration=40, iteration=3, product=120
(alarm-multiple) thread 1: duration=20, iteration=7, product=140
(alarm-multiple) thread 2: duration=30, iteration=5, product=150
(alarm-multiple) thread 4: duration=50, iteration=3, product=150
(alarm-multiple) thread 3: duration=40, iteration=4, product=160
(alarm-multiple) thread 2: duration=30, iteration=6, product=180
(alarm-multiple) thread 3: duration=40, iteration=5, product=200
(alarm-multiple) thread 4: duration=50, iteration=4, product=200
(alarm-multiple) thread 2: duration=30, iteration=7, product=210
(alarm-multiple) thread 3: duration=40, iteration=6, product=240
(alarm-multiple) thread 4: duration=50, iteration=5, product=250
(alarm-multiple) thread 3: duration=40, iteration=7, product=280
(alarm-multiple) thread 4: duration=50, iteration=6, product=300
(alarm-multiple) thread 4: duration=50, iteration=7, product=350
(alarm-multiple) end
Execution of 'alarm-multiple' complete.
Timer: 589 ticks
Thread: 0 idle ticks, 589 kernel ticks, 0 user ticks
Console: 2954 characters output
Keyboard: 0 keys pressed
Powering off...
root@e71af4c9d9b0:/home/pintos/threads# f10921a18
```

Reference

- Source code: https://github.com/WyldeCat/pintos-anon
- 中文文檔: https://docs.etao.net
- 英文文檔: https://web.stanford.edu/class/cs140/projects/pintos/pintos.html
- 英文 pdf 文檔: https://web.stanford.edu/class/cs140/projects/pintos/pintos.pdf

Thanks for your attention !

